JPA query language

학습목표

JPA query language 문법을 학습한다.
JPA query language를 사용하여 리포지토리 메소드를 구현한다.

목차

1. 배경지식 2
   1) 개요 2
   2) student2 예제 데이터베이스 3
2. JPA6 프로젝트 6
   1) 프로젝트 생성 6
   2) application.properties 8
3. entity 클래스 구현 #1 9
   1) Department.java 9
   2) Student.java 9
   3) Professor.java 10
   4) Lecture.java 10
   5) Sugang.java 11
4. entity 클래스 구현 #2 12
   1) Department.java 12
   2) Student.java 13
   3) Professor.java 14
   4) Lecture.java 15
5. JPQL 실습 환경 구현 16
   1) 개요 16
   2) JPQLController.java 16
   3) index.html 생성 17
   4) 실행 18
6. JPA query language 19
   1) 개요 19
   2) JPQL (JPA query language) 19
   3) 조인 방법 - single valued association 23
   4) 조인 방법 - collection valued association 24
   5) JPQL 예제 26
   6) Group By 29
   7) JPQL 함수 29
   8) UPDATE / DELETE 30
7. JPQL 메소드 구현 31
   1) StudentRepository.java 생성 31
   2) SugangRepository.java 33
   3) StudentController.java 생성 34
   4) SugangController.java 35
   5) 실행 36
   6) StudentRepository.java 수정 37
8. 부록 - 고급 SQL 38

1. 배경지식

1) 개요

JPA 고급 조회 기능을 사용하려면
  single valued association 뿐만 아니라
  collection valued association까지 다 구현해야 한다.

JPA 고급 조회 기능

- JPA query creation 고급 조회 기능
- JPA Query Language

JPA query creation 고급 조회 기능

백엔드 실무에서 DB 데이터를 조회할 때,
다수의 테이블을 조인해야 하는 복잡한 SQL 구현을 해야 하는 경우가 많다.

그런데 이런 복잡한 데이터 조회를
JPA query creation 고급 기능을 이용하여 쉽게 구현할 수도 있다.
그런데 먼저 collection valued association 구현을 해야 한다.

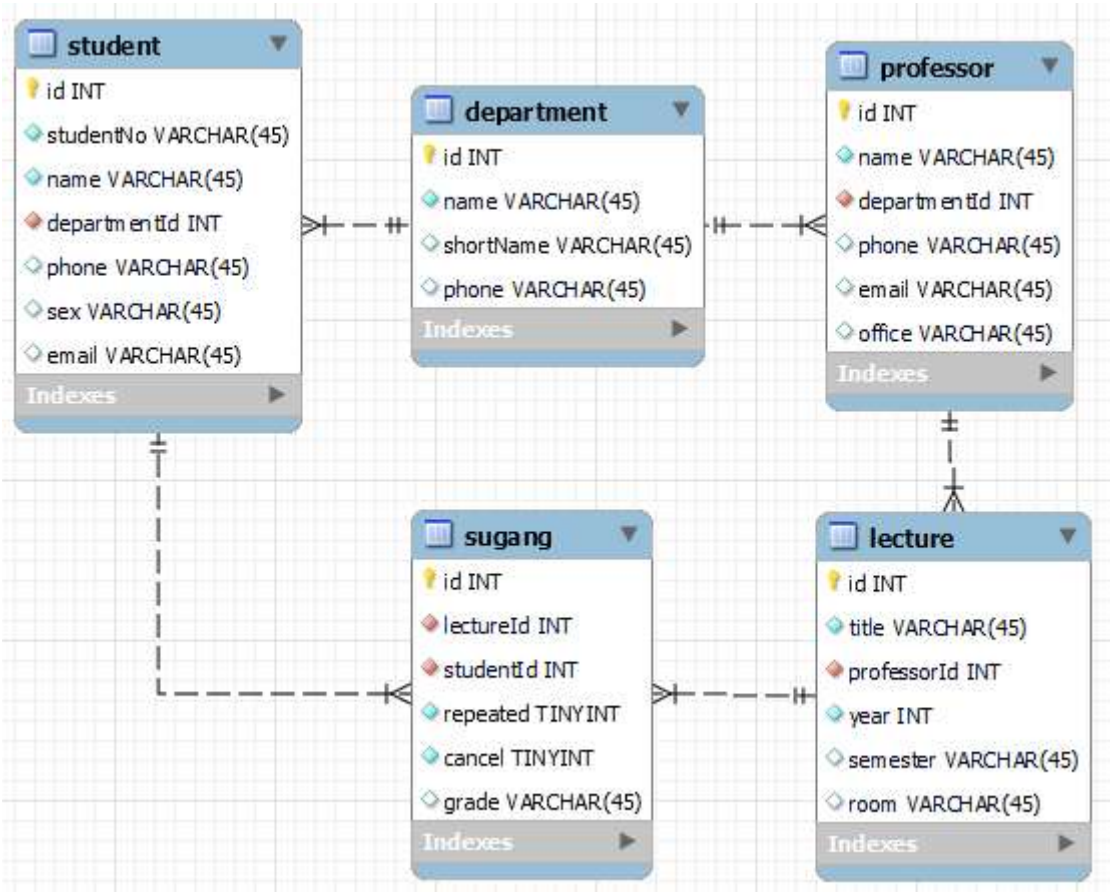
JPA Query Langauge

JPA에도 SQL 문법과 비슷한 query language가 있다.
그것이 JPA Query Language이고 줄여서 JPQL 이라고 한다.

복잡한 데이터 조회를 SQL 보다 훨씬 간단하게 구현할 수 있다.
그런데 먼저 collection valued association 구현을 해야 한다.

2) student2 예제 데이터베이스

student2 데이터베이스에서
single valued association과 collection valued association을 다 구현하자.



student - department 관계

student 테이블에 외래키가 있다. departmentId
학생이 소속된 학과

Student

```
    @ManyToOne
    @JoinColumn(name = "departmentId")
    Department department;
```

Department

```
    @OneToMany(mappedBy="department")
    List<Student> students;
```

professor - department 관계

professor 테이블에 외래키가 있다. departmentId
교수가 소속된 학과

Professor

```
    @ManyToOne
    @JoinColumn(name = "departmentId")
    Department department;
```

Department

```
    @OneToMany(mappedBy="department")
    List<Professor> professors;
```

lecture - professor 관계

lecture 테이블에 외래키가 있다. professorId
강의를 담당하는 교수

Lecture

```
    @ManyToOne
    @JoinColumn(name = "professorId")
    Professor professor;
```

Professor

```
    @OneToMany(mappedBy="professor")
    List<Lecture> lectures;
```

sugang - student 관계

sugang 테이블에 외래키가 있다. studentId
수강신청을 한 학생

Sugang

```
    @ManyToOne
    @JoinColumn(name = "studentId")
    Student student;
```

Student

```
    @OneToMany(mappedBy="student")
    List<Sugang> sugangs;
```

sugang - lecture 관계

sugang 테이블에 외래키가 있다. lectureId
수강신청 대상 강좌

Sugang

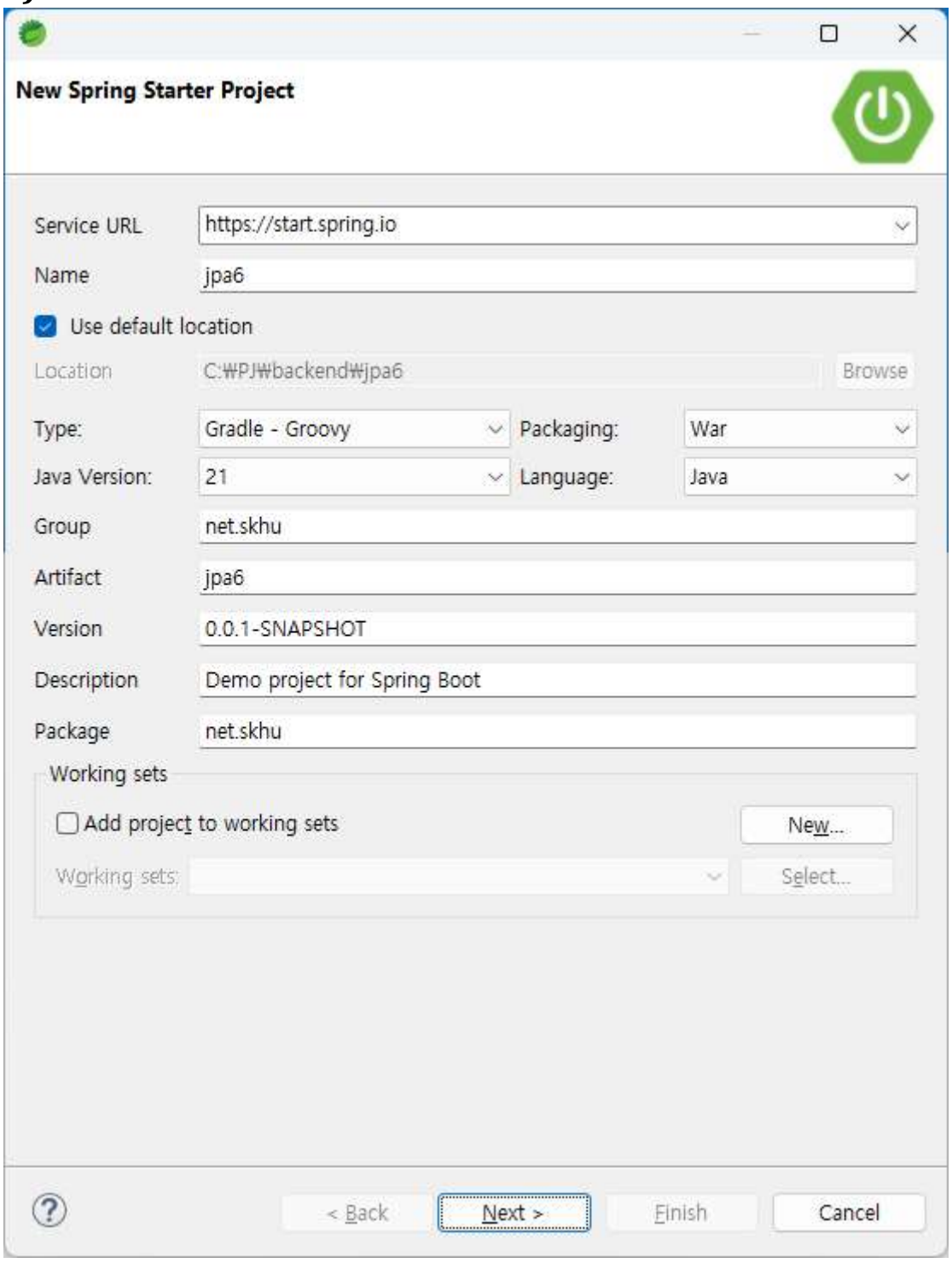
```
    @ManyToOne
    @JoinColumn(name = "lectureId")
    Lecture lecture;
```

Lecture

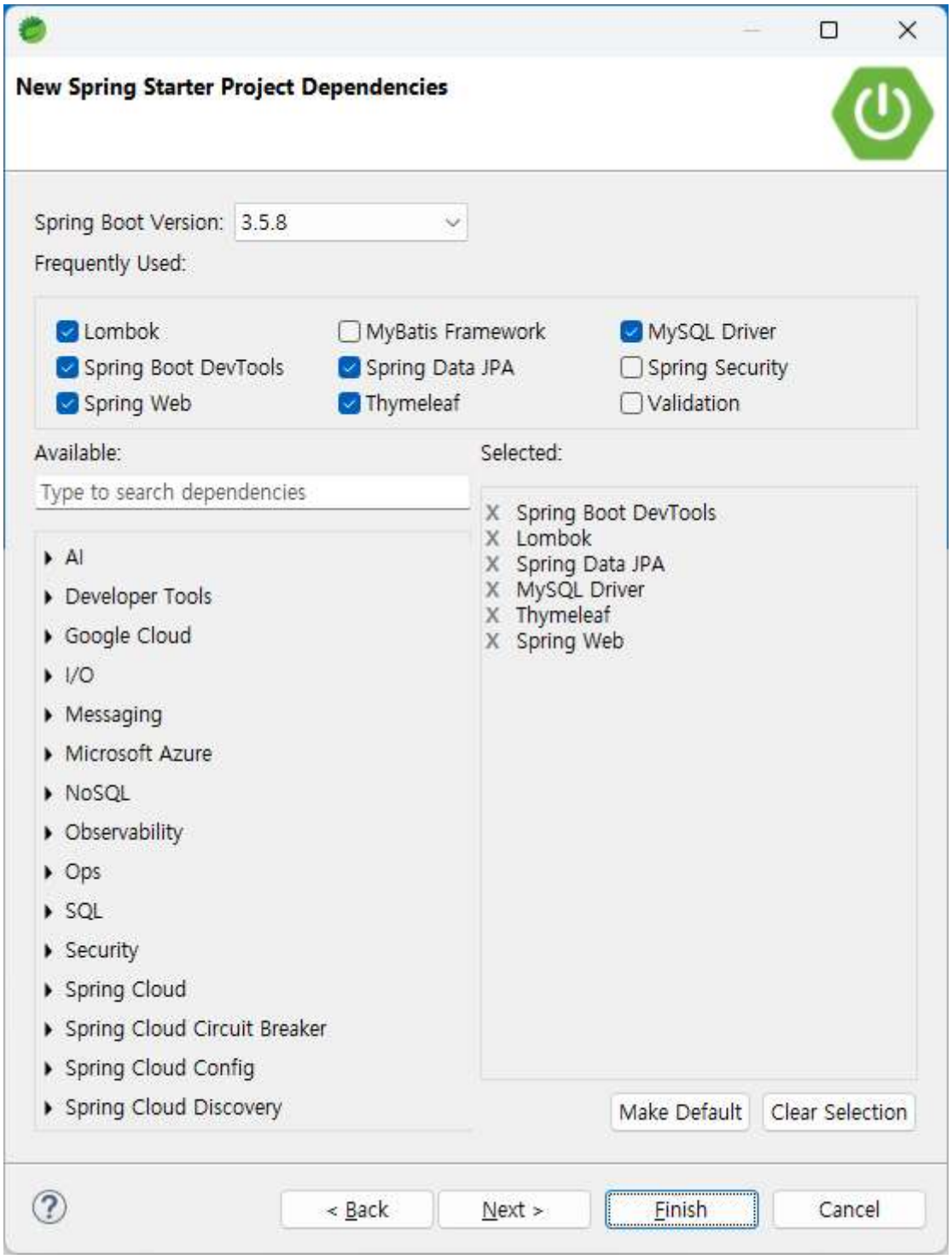
```
    @OneToMany(mappedBy="lecture")
    List<Sugang> sugangs;
```

2. JPA6 프로젝트

1) 프로젝트 생성



Name:	jpa5
Type:	Gradle - Groovy
Java Version:	21
Package:	next.skhu


New Spring Starter Project Dependencies
Spring Boot Version: 3.5.8
Frequently Used:
Lombok
MyBatis Framework
MySQL Driver
Spring Boot DevTools
Spring Data JPA
Spring Security
Spring Web
Thymeleaf
Validation
Available:
Type to search dependencies
AI
Developer Tools
Google Cloud
I/O
Messaging
Microsoft Azure
NoSQL
Observability
Ops
SQL
Security
Spring Cloud
Spring Cloud Circuit Breaker
Spring Cloud Config
Spring Cloud Discovery
Selected:
X Spring Boot DevTools
X Lombok
X Spring Data JPA
X MySQL Driver
X Thymeleaf
X Spring Web
Make Default
Clear Selection
< Back
Next >
Finish
Cancel

2) application.properties

```
spring.application.name=jpa6
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/student2?useUnicode=yes&characterEncoding=UTF-8&allowMultiQueries=true&serverTimezone=Asia/Seoul
spring.datasource.username=user1
spring.datasource.password=skhuA+4.5
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptog.sql.BasicBinder=TRACE
server.port=8088
```

3. entity 클래스 구현 #1

먼저 single-valued association만 구현하자.

1) Department.java

src/main/java/net/skhu/entity/Department.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Data;

@Data
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String name;
    String shortName;
    String phone;
}
```

2) Student.java

src/main/java/net/skhu/entity/Student.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import lombok.Data;

@Data
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String studentNo;
    String name;
    String phone;
    String sex;
    String email;

    @ManyToOne
    @JoinColumn(name = "departmentId")
    Department department;
}
```

3) Professor.java

src/main/java/net/skhu/entity/Professor.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import lombok.Data;

@Data
@Entity
public class Professor {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String name;
    String phone;
    String email;
    String office;

    @ManyToOne
    @JoinColumn(name = "departmentId")
    Department department;
}
```

4) Lecture.java

src/main/java/net/skhu/entity/Lecture.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import lombok.Data;

@Data
@Entity
public class Lecture {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String title;
    int year;
    String room;
    String semester;

    @ManyToOne
    @JoinColumn(name = "professorId")
    Professor professor;
}
```

5) Sugang.java

src/main/java/net/skhu/entity/Sugang.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import lombok.Data;

@Data
@Entity
public class Sugang {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    boolean repeated;
    boolean cancel;
    String grade;

    @ManyToOne
    @JoinColumn(name = "studentId")
    Student student;

    @ManyToOne
    @JoinColumn(name = "lectureId")
    Lecture lecture;
}
```

4. entity 클래스 구현 #2

collection-valued association을 추가하자.

1) Department.java

src/main/java/net/skhu/entity/Department.java

```java
package net.skhu.entity;

import java.util.List;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;

@Data
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String name;
    String shortName;
    String phone;

    @JsonIgnore
    @ToString.Exclude
    @EqualsAndHashCode.Exclude
    @OneToMany(mappedBy="department")
    List<Student> students;

    @JsonIgnore
    @ToString.Exclude
    @EqualsAndHashCode.Exclude
    @OneToMany(mappedBy="department")
    List<Professor> professors;
}
```

2) Student.java

src/main/java/net/skhu/entity/Student.java

```java
package net.skhu.entity;

import java.util.List;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.OneToMany;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;

@Data
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String studentNo;
    String name;
    String phone;
    String sex;
    String email;

    @ManyToOne
    @JoinColumn(name = "departmentId")
    Department department;

    @JsonIgnore
    @ToString.Exclude
    @EqualsAndHashCode.Exclude
    @OneToMany(mappedBy="student")
    List<Sugang> sugangs;

}
```

3) Professor.java

src/main/java/net/skhu/entity/Professor.java

```java
package net.skhu.entity;

import java.util.List;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.OneToMany;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;

@Data
@Entity
public class Professor {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String name;
    String phone;
    String email;
    String office;

    @ManyToOne
    @JoinColumn(name = "departmentId")
    Department department;

    @JsonIgnore
    @ToString.Exclude
    @EqualsAndHashCode.Exclude
    @OneToMany(mappedBy="professor")
    List<Lecture> lectures;
}
```

4) Lecture.java

src/main/java/net/skhu/entity/Lecture.java

```java
package net.skhu.entity;

import java.util.List;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.OneToMany;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;

@Data
@Entity
public class Lecture {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String title;
    int year;
    String room;
    String semester;

    @ManyToOne
    @JoinColumn(name = "professorId")
    Professor professor;

    @JsonIgnore
    @ToString.Exclude
    @EqualsAndHashCode.Exclude
    @OneToMany(mappedBy="lecture")
    List<Sugang> sugangs;
}
```

5. JPQL 실습 환경 구현

1) 개요

JPQL을 입력하고 실행해 볼 수 있는 간단한 웹페이지를 구현하려고 한다.
이 구현은 이 수업의 학습 목표가 아니다.

2) JPQLController.java

src/main/java/net/skhu/controller/JPQLController.java

```java
package net.skhu.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.jpa.repository.JpaContext;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;
import jakarta.persistence.EntityManager;
import jakarta.persistence.Query;
import net.skhu.entity.Student;

@Controller
public class JPQLController {

    @Autowired JpaContext jpaContext;

    @GetMapping("jpql")
    public String index() {
        return "jpql/index";
    }

    @GetMapping(value="jpql", params="q")
    public @ResponseBody Object jpql(@RequestParam("q") String q) {
        EntityManager manager = jpaContext.getEntityManagerByManagedType(Student.class);
        Query query = manager.createQuery(q);
        Object r = query.getResultList();
        return r;
    }
}
```

query string에 아무것도 없이 "jpql" URL을 요청하면 다음 액션 메소드가 호출된다.

```java
    @GetMapping("jpql")
    public String index() {
```

query string에 jpql 항목을 포함하여 "jpql" URL을 요청하면 다음 액션 메소드가 호출된다.

```java
    @GetMapping(value="jpql", params="q")
    public @ResponseBody Object jpql(@RequestParam("q") String jpql) {
```

3) index.html 생성

src/main/resources/templates/index.html

```
<!doctype html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<script src="https://cdn.jsdelivr.net/npm/vue/dist/vue.js"></script>
<script src="https://unpkg.com/axios/dist/axios.min.js"></script>
<style>
  table { border-collapse: collapse; margin-top: 5px; }
  th, td { border: 1px solid gray; padding: 4px; }
  div#msg { padding: 5px; font-size: 12pt; }
</style>
</head>
<body>
<div id="app">
  <form>
    <textarea rows="10" cols="100" v-model="jpql"></textarea> <br />
    <button type="button" @click="search">JPQL 실행</button>
  </form>
  <div v-if="error" class="msg">
    {{ error }} <br /> {{ error?.response?.data?.message }}
  </div>
  <table v-if="!error && (typeof list[0] == 'object')">
    <thead>
      <tr><th v-for="(value, key) in list[0]">{{ key }}</th></tr>
    </thead>
    <tbody>
      <tr v-for="obj in list">
        <td v-for="value in obj">{{ value }}</td>
      </tr>
    </tbody>
  </table>
  <ul v-if="!error && (typeof list[0] != 'object')">
     <li v-for="value in list">{{ value }}</li>
  </ul>
</div>
<script type="text/javascript">
  var app = new Vue({
    el: '#app',
    data: {
      jpql: "",
      list: [],
      error: null
    },
    methods: {
      search: function() {
        this.list = null;
        this.error = null;
        var params = { q: this.jpql };
        axios.get(location.href, { params: params })
             .then( (response) => this.list = response.data )
             .catch( (error) => this.error = error );
      }
    }
  })
</script>
</html>
```

간단한 Vue.js 프론트엔드 구현

4) 실행

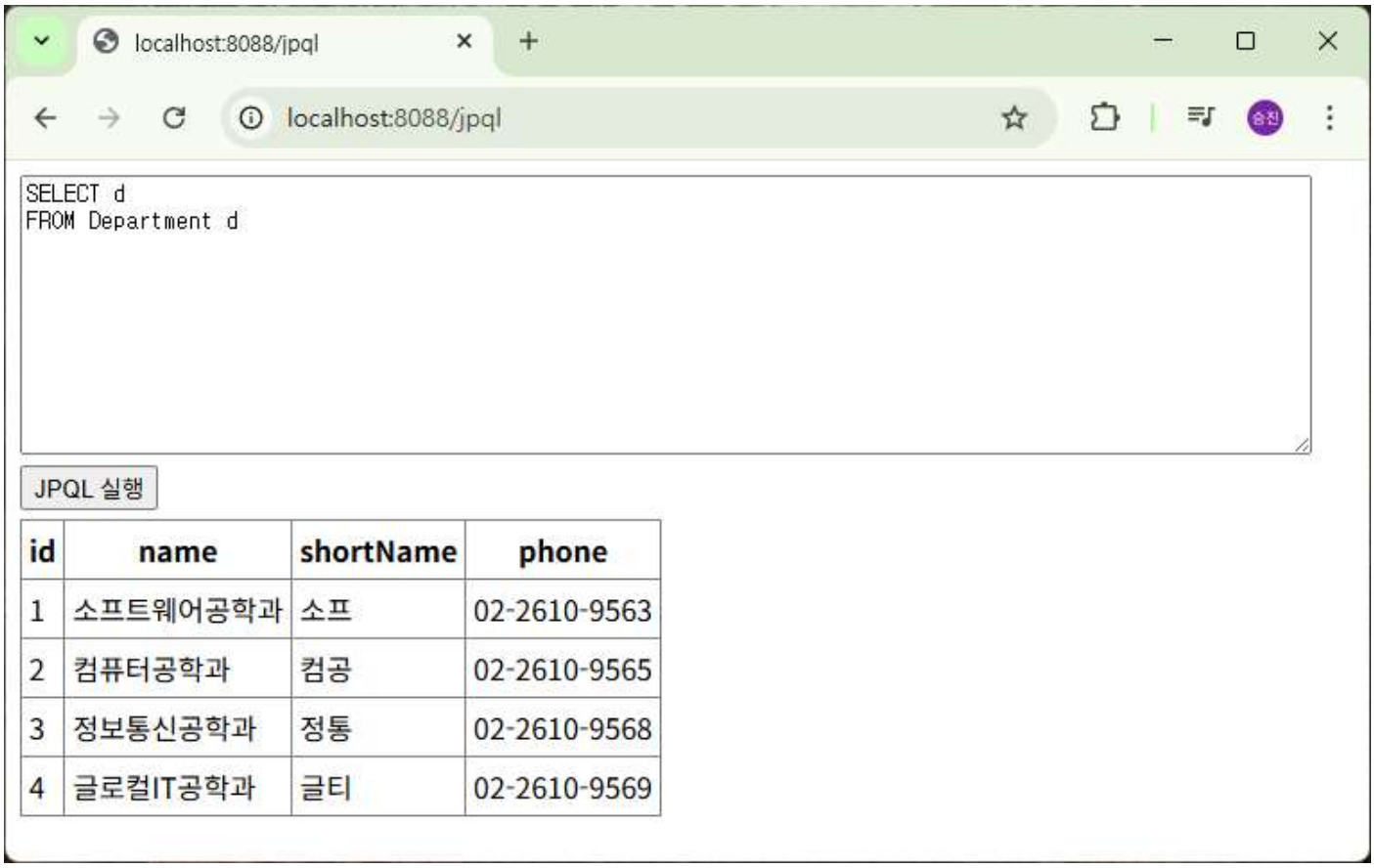
http://localhost:8088/jpql



위 화면에 JPQL 명령을 입력하고 실행 버튼을 클릭하면,
JPQL 조회 결과가 출력된다.

예:
JPQL 명령을 입력하고 실행 버튼을 클릭

```
SELECT d
FROM Department d
```



id	name	shortName	phone
1	소프트웨어공학과	소프	02-2610-9563
2	컴퓨터공학과	컴공	02-2610-9565
3	정보통신공학과	정통	02-2610-9568
4	글로컬IT공학과	글티	02-2610-9569

6. JPA query language

1) 개요

JPA query language 문법 즉 JPQL 문법을 설명하려고 한다.

그런데, 데이터 조회를 구현할 때 JPA query creation 기능으로 구현 가능한 경우가 많다.
아래 JPQL 문법 설명을 위한 간단한 JPQL 예제들은 대부분
JPA query creation 기능만으로 구현이 가능하다.

JPA query creation 기능만으로 구현이 불가능할 때 JPA query language 구현을 시도하자.

2) JPQL (JPA query language)

JPQL 문법은 SQL 문법과 유사하다.
JPQL 은 엔터티 객체들을 조회한다.

예1

JPQL

```
SELECT d
FROM Department d
```

위 JPQL은 List<Department> 객체를 리턴한다.
전체 Department 객체 목록을 리턴한다.

예2

JPQL

```
SELECT s
FROM Student s
WHERE s.studentNo = '201532006'
```

JPA query creation

```
List<Student> findByStudentNo(String studentNo);
```

예3

JPQL

```
SELECT s
FROM Student s
WHERE s.department.id = 1
```

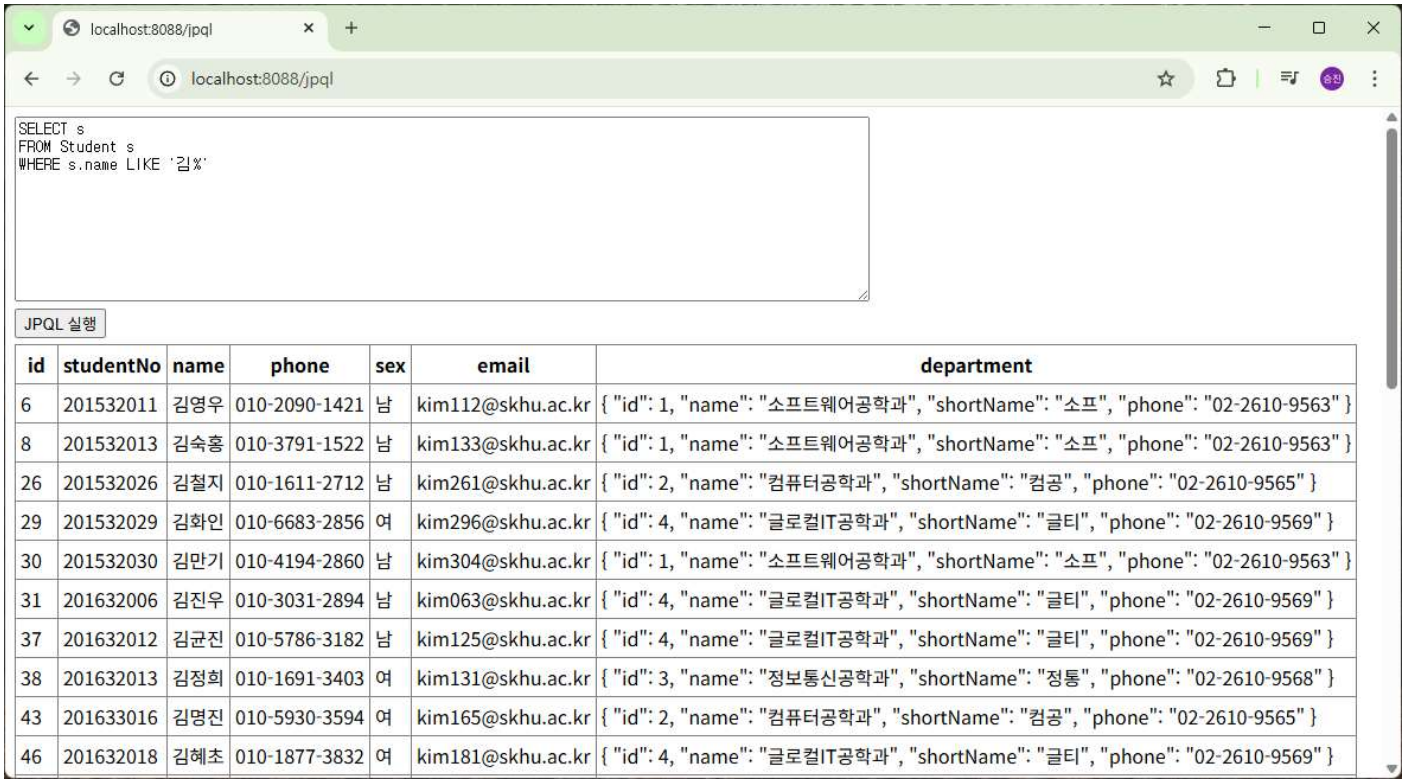
주의: Student 엔터티 클래스에는 departmentId 속성이 없고, 대신 department 속성이 있다.

JPA query creation

```
List<Student> findByDepartmentId(int departmentId);
```

예3

```
SELECT s
FROM Student s
WHERE s.name LIKE '김%'
```



id	studentNo	name	phone	sex	email	department
6	201532011	김영우	010-2090-1421	남	kim112@skhu.ac.kr	{ "id": 1, "name": "소프트웨어공학과", "shortName": "소프", "phone": "02-2610-9563" }
8	201532013	김숙홍	010-3791-1522	남	kim133@skhu.ac.kr	{ "id": 1, "name": "소프트웨어공학과", "shortName": "소프", "phone": "02-2610-9563" }
26	201532026	김철지	010-1611-2712	남	kim261@skhu.ac.kr	{ "id": 2, "name": "컴퓨터공학과", "shortName": "컴공", "phone": "02-2610-9565" }
29	201532029	김화인	010-6683-2856	여	kim296@skhu.ac.kr	{ "id": 4, "name": "글로컬IT공학과", "shortName": "글티", "phone": "02-2610-9569" }
30	201532030	김만기	010-4194-2860	남	kim304@skhu.ac.kr	{ "id": 1, "name": "소프트웨어공학과", "shortName": "소프", "phone": "02-2610-9563" }
31	201632006	김진우	010-3031-2894	남	kim063@skhu.ac.kr	{ "id": 4, "name": "글로컬IT공학과", "shortName": "글티", "phone": "02-2610-9569" }
37	201632012	김균진	010-5786-3182	남	kim125@skhu.ac.kr	{ "id": 4, "name": "글로컬IT공학과", "shortName": "글티", "phone": "02-2610-9569" }
38	201632013	김정희	010-1691-3403	여	kim131@skhu.ac.kr	{ "id": 3, "name": "정보통신공학과", "shortName": "정통", "phone": "02-2610-9568" }
43	201633016	김명진	010-5930-3594	여	kim165@skhu.ac.kr	{ "id": 2, "name": "컴퓨터공학과", "shortName": "컴공", "phone": "02-2610-9565" }
46	201632018	김혜초	010-1877-3832	여	kim181@skhu.ac.kr	{ "id": 4, "name": "글로컬IT공학과", "shortName": "글티", "phone": "02-2610-9569" }

JPA query creation

```
List<Student> findByNameStartsWith(String name);
```

예4 JPQL

```
SELECT s
FROM Student s
WHERE s.department.id = 1
OR s.department.id = 2
ORDER BY s.name
```

```
SELECT s
FROM Student s
WHERE s.department.id IN (1, 2)
ORDER BY s.name DESC
```

예4 JPA query creation

StudentRepository.java

```
package net.skhu.repository;

import java.util.Collection;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import net.skhu.entity.Student;

public interface StudentRepository extends JpaRepository<Student, Integer>  {
    List<Student> findByDepartmentIdIn(Collection<Integer> departmentIds);
}
```

StudentController.java

```
package net.skhu.controller;

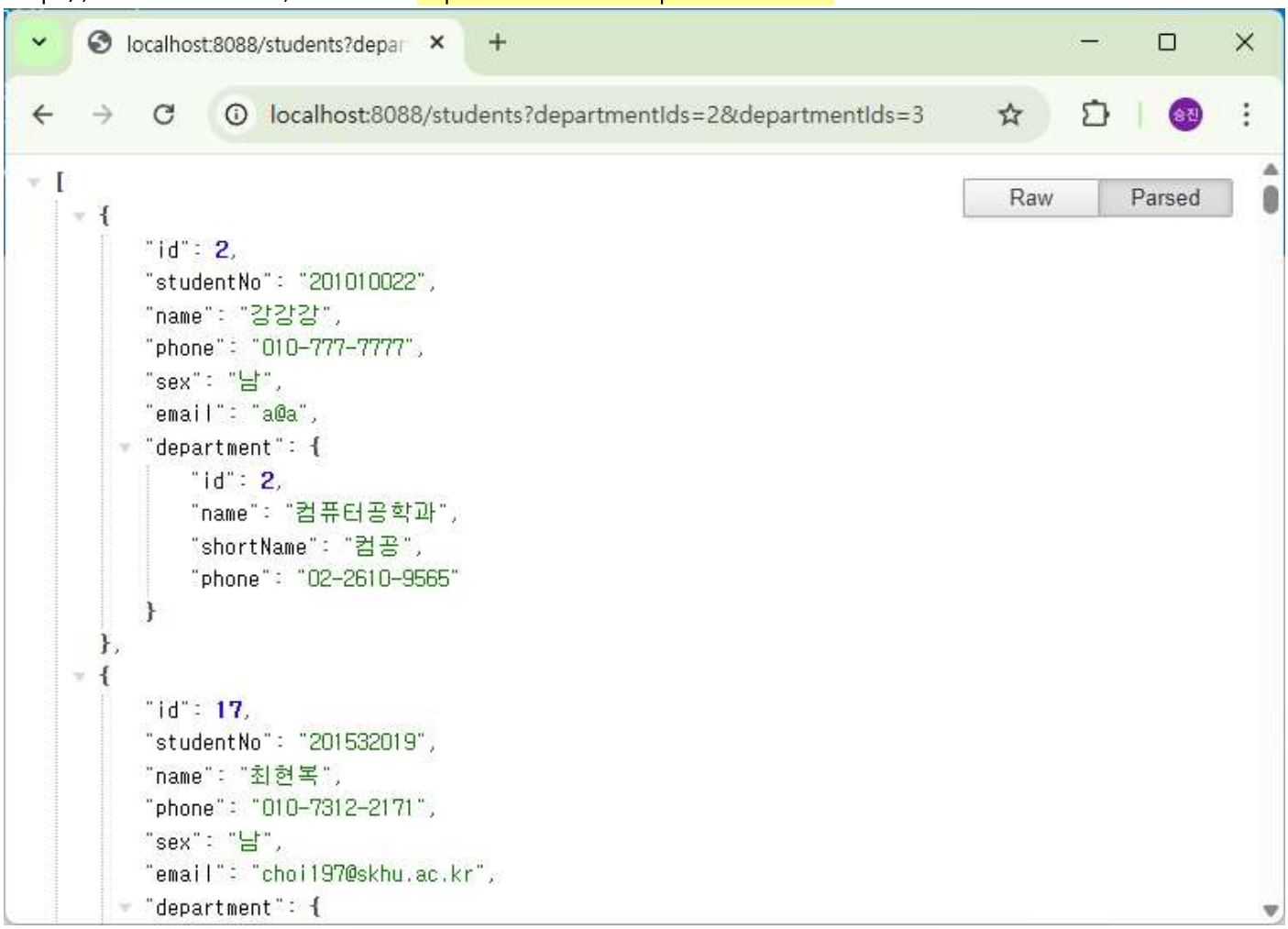
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import net.skhu.entity.Student;
import net.skhu.repository.StudentRepository;

@RestController
@RequestMapping("students")
public class StudentController {

    @Autowired StudentRepository studentRepository;

    @GetMapping(value="",params="departmentIds")
    public List<Student> getByDepartmentIds(
          @RequestParam("departmentIds") List<Integer> departmentIds)
    {
        return studentRepository.findByDepartmentIdIn(departmentIds);
    }
}
```

http://localhost:8088/students?departmentIds=2&departmentIds=3



3) 조인 방법 - single valued association

SQL

학생 목록을 조회할 때, 조회 결과에 학과명을 포함하려면
Student 테이블과 Department 테이블을 조인해야 한다.
예를 들어, 학번, 이름, 학과명 목록을 조회하는 SQL은 다음과 같다.

```
SELECT s.studentNo, s.name, d.name
FROM student s LEFT JOIN department d
   ON s.departmentId = d.id
```

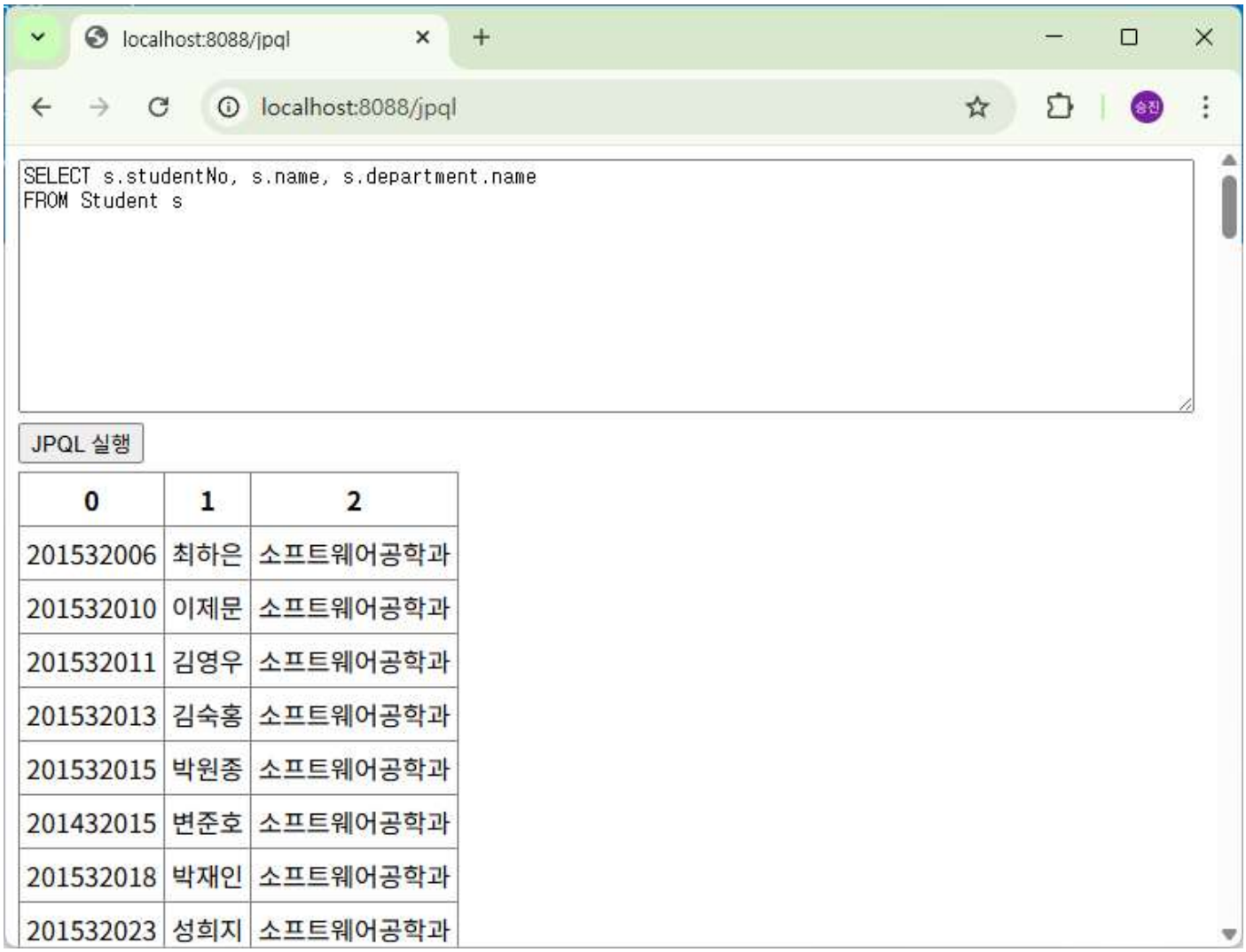
JPQL

JPA에서는 조인하지 않고도 구현할 수 있는 경우가 많다.
학번, 이름, 학과명 목록을 조회하는 JPQL은 다음과 같다.

```
SELECT s.studentNo, s.name, s.department.name
FROM Student s
```

참고
Student 엔터티 클래스에는 department 속성이 포함되어 있다.

```
@ManyToOne
@JoinColumn(name = "departmentId")
Department department;
```



0	1	2
201532006	최하은	소프트웨어공학과
201532010	이제문	소프트웨어공학과
201532011	김영우	소프트웨어공학과
201532013	김숙홍	소프트웨어공학과
201532015	박원종	소프트웨어공학과
201432015	변준호	소프트웨어공학과
201532018	박재인	소프트웨어공학과
201532023	성희지	소프트웨어공학과

4) 조인 방법 - collection valued association

SQL

'201532006' 학번 학생이 수강(sugang)하고 있는 강좌(lecture) 이름을 조회하는 SQL 명령은 다음과 같다.

```
SELECT t.title
FROM student s
  JOIN sugang g ON s.id = g.studentId
  JOIN lecture t ON g.lectureId = t.id
WHERE s.studentNo = '201532006'
```

JPQL

'201532006'학번 학생이 수강(sugang)하고 있는 강좌(lecture) 이름을 조회하는 JPQL 명령은 다음과 같다.

```
SELECT g.lecture.title
FROM Student s JOIN s.sugangs g
WHERE s.studentNo = '201532006'
```

참고
Student 엔터티 클래스에는 sugangs 속성이 포함되어 있다.

```
@OneToMany(mappedBy="student")
List<Sugang> sugangs;
```

Sugang 엔터티 클래스에는 lecture 속성이 포함되어 있다.

```
Lecture lecture;
```

아래처럼 구현해도 된다.

```
SELECT g.lecture.title
FROM Sugang g
WHERE g.student.studentNo = '201532006'
```

collection-valued association에서 JOIN을 해야 하는 이유

```
Sugang sugang = sugangRepository.findById(1);
System.out.print(sugang.getGrade());  // OK

List<Sugang> sugangs = sugangRepository.findAll();
System.out.print(sugangs.getGrade()); // 에러: sugangs 객체는 Sugang 클래스가 아니고 List<Sugang>
이다.
                                      // List<Sugang> 에는 getGrade() 메소드가 없다.

SELECT s.sugangs.grade  // 에러: sugangs 객체에는 grade 속성이 없다.
FROM Student s
WHERE s.studentNo = '201532006'

SELECT g.grade
FROM Student s JOIN s.sugangs g
WHERE s.studentNo = '201532006'
```

5) JPQL 예제

```
SELECT d
FROM Department d
```

Department 객체 목록을 리턴한다.

```
SELECT d.students
FROM Department d
WHERE d.id = 1
```

id 값이 1인 학과(Department) 소속 학생(Student) 목록을 리턴한다.

Department 클래스에 students 멤버 변수가 포함되어 있다.
students 멤버 변수는, 학과 소속 학생 목록에 해당하고, 학과와 학생 사이의 1 대 다 관계를 구현한다.

```
SELECT s
FROM Department d JOIN d.students s
WHERE d.id = 1
```

id 값이 1인 학과(Department) 소속 학생(Student) 목록을 리턴한다.

```
SELECT s.studentNo, s.name, d.name
FROM Department d JOIN d.students s
WHERE d.id = 1
```

```
SELECT g.lecture.title
FROM Student s JOIN s.sugangs g
WHERE s.id = 1
```

id 값이 1인 학생이 수강하는 강좌의 이름 목록을 리턴한다.

```
SELECT s
FROM Student s
WHERE SIZE(s.sugangs) > 4
```

수강의 수가 4를 초과하는 학생 목록

```
SELECT s
FROM Student s
WHERE SIZE(s.sugangs) = 0
```

수강 강좌가 없는 학생 목록

```
SELECT s
FROM Student s
WHERE s.sugangs IS EMPTY
```

```
SELECT s
FROM Student s
WHERE EXISTS (SELECT g
              FROM s.sugangs g
              WHERE g.lecture.professor.name = '이몽룡')
```

'이몽룡' 교수의 과목을 하나라도 수강하는 학생 목록

```
SELECT s
FROM Student s JOIN s.sugangs g
WHERE g.lecture.title = '자료구조'
```

자료구조 과목을 수강하는 학생 목록 - JPQL 구현

```
StudentRepository
     List<Student> findBySugangsLectureTitle(String title);

List<Student> students = studentRepository.findBySugangsLectureTitle("자료구조");
```

자료구조 과목을 수강하는 학생 목록 - query creation 구현

'박남기' 학생이 수강하는 강좌들의 담당교수 목록

```

SELECT g.lecture.professor
FROM Student s JOIN s.sugangs g
WHERE s.name = '박남기'

```

'박남기' 학생이 수강하는 강좌명 목록 - JPQL

```
SELECT g.lecture.title
FROM Student s JOIN s.sugangs g
WHERE s.name = '박남기'

SELECT g.lecture.title
FROM Sugang g
WHERE g.student.name = '박남기'

SELECT t.title
FROM Lecture t JOIN t.sugangs g
WHERE g.student.name = '박남기'
```

'박남기' 학생이 수강하는 강좌명 목록 - SQL 문법

```
use student2;

SELECT t.title
FROM student s
  JOIN sugang g ON g.studentId = s.id
  JOIN lecture t ON g.lectureId = t.id
WHERE s.name = '박남기'
```

'박남기' 학생이 수강하는 강좌의 강좌명과 담당교수 목록 - JPQL

```
SELECT g.lecture.title, g.lecture.professor.name
FROM Student s JOIN s.sugangs g
WHERE s.name = '박남기'

SELECT g.lecture.title, g.lecture.professor.name
FROM Sugang g
WHERE g.student.name = '박남기'

SELECT c.name, c.professor.name
FROM Lecture c JOIN c.sugangs g
WHERE g.student.name = '박남기'
```

'박남기' 학생이 수강하는 강좌의 강좌명과 담당교수 목록 - SQL

```
use student2;

SELECT t.title, p.name
FROM student s
  JOIN sugang g ON g.studentId = s.id
  JOIN lecture t ON g.lectureId = t.id
  JOIN professor p ON t.professorId = p.id
WHERE s.name = '박남기'
```

(학번, 이름, 수강 수) 조회 - 수강 수 내림차순

```
SELECT s.studentNo, s.name, SIZE(s.sugangs)
FROM Student s
ORDER BY SIZE(s.sugangs) DESC
```

```
SELECT g
FROM Sugang g
WHERE g.lecture.id = 3 AND g.grade = 'C0'
```

6) Group By

```
SELECT s.department, COUNT(s)
FROM Student s
GROUP BY s.department
```

학과별 학생 수를 조회한다.

```
SELECT g.lecture.title, COUNT(g)
FROM Sugang g
GROUP BY g.lecture
```

강좌별 학생 수를 조회한다.

7) JPQL 함수

JPQL 명령에서 사용할 수 있는 함수

ABS(*number*)	절대값을 리턴한다
CONCAT(*string1*, *string2*)	두 문자열을 연결하여 리턴한다
CURRENT_DATE()	현재 날짜를 리턴
CURRENT_TIME()	현재 시각을 리턴
CURRENT_TIMESTAMP()	현재 날짜 시각을 리턴
INDEX(*identification variable*)	identification variable이 참조하는 엔터티의, 컬렉션(collection)에서 인덱스를 리턴
LENGTH(*string*)	문자열 길이를 리턴
LOCATE(*string1*, *string2* [, *start*])	string2 문자열에서 string1 문자열의 위치를 리턴한다. start 값이 주어졌으면 start 위치부터 찾기 시작한다. start1 문자열이 start2 문자열의 시작 부분과 일치하면 1을 리턴한다. 일치하는 부분이 없으면 0을 리턴한다.
LOWER(*string*)	문자열을 소문자로 변환하여 리턴한다.
MOD(*number1*, *number2*)	number1을 number2로 나눈 나머지를 리턴한다.
SIZE(*collection*)	컬렉션(collection)의 크기를 리턴한다. 컬렉션에 포함된 객체의 수를 리턴.
SQRT(*number*)	제곱근을 리턴한다.
SUBSTRING(*string*, *start*, *end*)	부분 문자열을 리턴한다.
UPPER(*string*)	문자열을 대문자로 변환하여 리턴한다.
TRIM(LEADING FROM *string*)	문자열 앞 부분의 공백을 제거하고 리턴한다.
TRIM(TRAILING FROM *string*)	문자열 뒷 부분의 공백을 제거하고 리턴한다.
TRIM(BOTH FROM *string*)	문자열 앞 뒤의 공백을 제거하고 리턴한다.
TRIM(*string*)	문자열 앞 뒤의 공백을 제거하고 리턴한다.

8) UPDATE / DELETE

SELECT 뿐만 아니라, UPDATE, DELETE 명령도 구현할 수 있다.

JPA Repository의 기본 메소드 save 가 충분하기 때문에, INSERT 명령은 필요 없다.
그래서 JPQL 에 INSERT 명령은 없다.

JPA repository 메소드 이름이 정해진 규칙에 맞으면, 메소드를 자동으로 구현해주는
query create 기능이 충분히 강력하기 때문에,
DELETE JPQL 명령을 사용할 일도 거의 없다.

JPQL query를 직접 구현해야 하는 경우는, 대부분 SELECT, UPDATE 명령이다.

여러 레코드를 일괄 업데이트 해야 할 때, JPA Repository의 기본 메소드 save 를 반복 호출하는 것은 불편하다.

```
UPDATE Sugang g
SET g.grade = 'D+'
WHERE g.lecture.id = 3 AND g.grade = 'C0'
```

lecture id 3 강좌의 C0 성적을 전부 D+ 로 수정한다.

이렇게 데이터를 수정하는 JPQL은 앞에서 구현한 JPQL 실습 컨트롤러로 실행할 수 없다.
앞에서 구현한 JPQL 컨트롤러는 조회 명령만 실행 할 수 있다.

query creation 메소드의 예

```
void delete(int id);
void deleteByName(String name);
void deleteByStudentNo(String studentNo);
void deleteByNameOrStudentNo(String name, String studentNo);
void deleteByDepartmentName(String departmentName);
void deleteByDepartmentId(int departmentId);
```

위 메소드들의 이름이 JPA query creation 규칙에 맞기 때문에, 메소드가 자동으로 구현된다.

7. JPQL 메소드 구현

JPA repository에 JPQL 메소드를 구현하자.

1) StudentRepository.java 생성

src/main/java/net/skhu/repository/StudentRepository.java

```
1  package net.skhu.repository;
2
3  import java.util.List;
4  import org.springframework.data.jpa.repository.JpaRepository;
5  import org.springframework.data.jpa.repository.Query;
6  import net.skhu.entity.Student;
7
8  public interface StudentRepository extends JpaRepository<Student, Integer>  {
9
10         @Query("SELECT  s.studentNo,  s.name,  SIZE(s.sugangs)  FROM  Student  s  ORDER  BY
11 SIZE(s.sugangs) DESC")
12     List<Object[]> findSugangCount();
13
14     @Query("SELECT s FROM Student s JOIN s.sugangs g WHERE g.lecture.title = ?1")
15     List<Student> findByLectureTite(String title);
16
17     @Query("""
18          SELECT s
19          FROM Student s
20          WHERE EXISTS (SELECT g.lecture
21                        FROM s.sugangs g
22                           WHERE g.lecture.professor.id = ?1 OR g.lecture.professor.name = ?2)
23 """)
24     List<Student> findByProfessorIdOrProfessorName(int id, String name);
25
26     @Query("""
27          SELECT s
28          FROM Student s
29          WHERE EXISTS (SELECT g.lecture
30                         FROM s.sugangs g
31                              WHERE g.lecture.professor.id = :id OR g.lecture.professor.name =
32 :name) """)
       List<Student> findByProfessorNameOrProfessorId(String name, int id);

       }
```

```
@Query("SELECT  s.studentNo,  s.name,  SIZE(s.sugangs)  FROM  Student  s  ORDER  BY  SIZE(s.sugangs)
DESC")
List<Object[]> findSugangCount();
```

학번, 이름, 수강 수 목록을 조회한다. 수강 수 내림차순.
조회 결과 각각의 레코드를 Object[] 배열에 저장한다. (학번, 이름, 수강 수) 배열.

```
@Query("SELECT s FROM Student s JOIN s.sugangs g WHERE g.lecture.title = ?1")
List<Student> findByLectureTite(String title);
```

어떤 강좌를 수강하는 학생 목록을 조회한다.
JPQL 에서 ?1 부분에 메소드 첫째 파라미터 값이 삽입되어 실행된다.

```
@Query("""
         SELECT s
         FROM Student s
         WHERE EXISTS (SELECT g.lecture
                       FROM s.sugangs g
                       WHERE g.lecture.professor.id = ?1 OR g.lecture.professor.name = ?2) """)
List<Student> findByProfessorIdOrProfessorName(int id, String name);
```

어떤 교수의 강좌를 수강하는 학생 목록을 조회한다.
JPQL 에서 ?1 ?2 부분에 각각 메소드 첫째 파라미터 값, 둘때 파라미터 값이 삽입되어 실행된다.

```
@Query("""
         SELECT s
         FROM Student s
         WHERE EXISTS (SELECT g.lecture
                       FROM s.sugangs g
                       WHERE g.lecture.professor.id = :id OR g.lecture.professor.name = :name)
""")
List<Student> findByProfessorNameOrProfessorId(String name, int id);
```

어떤 교수의 강좌를 수강하는 학생 목록을 조회한다.
JPQL 에서 :id 부분에 id 파라미터 값이, :name 부분에 name 파라미터 값이 삽입되어 실행된다.

2) SugangRepository.java

src/main/java/net/skhu/repository/SugangRepository.java

```java
package net.skhu.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import net.skhu.entity.Sugang;

public interface SugangRepository extends JpaRepository<Sugang, Integer>  {

    List<Sugang> findByLectureIdOrderByGradeDesc(int lectureId);

    @Modifying(clearAutomatically = true, flushAutomatically = true)
    @Query("""
          UPDATE Sugang g SET g.grade = :after
          WHERE g.lecture.id = :lectureId AND g.grade = :before """)
    void udateGradeByLectureId(int lectureId, String before, String after);

}
```

```
List<Sugang> findByLectureIdOrderByGradeDesc(int lectureId);
  query creation 규칙에 맞기 때문에 메소드가 자동 구현된다.
  query creation 규칙으로 구현할 수 있는 이런 메소드는 JPQL로 구현할 필요 없다.


@Modifying(clearAutomatically = true, flushAutomatically = true)
@Query("""
      UPDATE Sugang g SET g.grade = :after
      WHERE g.lecture.id = :lectureId AND g.grade = :before """)
    void udateGradeByLectureId(int lectureId, String before, String after);
lecture id 강좌의 before 등급의 점수 전체를 after 등급으로 수정한다.

UPDATE / DELETE 명령인 경우에
@Modifying 어노테이션을 반드시 붙여야 한다.

clearAutomatically = true
  JPA 내부의 캐시를 비우라는 설정이다.
  UPDATE 명령이 DB의 데이터를 수정했기 때문에, JPA 내부 캐시의 데이터와 DB 데이터가 불일치 할 수 있다.

flushAutomatically = true
  캐시에 있는 내용 중에서 디스크에 저장되어야할 부분이 있다면, 바로 저장하라는 설정이다.
  디스크 저장 속도를 향상하는 수단으로 쓰기 캐시를 활용한다.
  디스크에 저장해야할 내용을 바로 저장하지 않고, 쓰기 캐시에 모으다가,
  쓰기 캐시가 꽉차면 한 번에 몰아서 디스크에 저장한다.
```

3) StudentController.java 생성

src/main/java/net/skhu/controller/StudentController.java

```java
package net.skhu.controller;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import net.skhu.entity.Student;
import net.skhu.repository.StudentRepository;

@RestController
@RequestMapping("student")
public class StudentController {

    @Autowired StudentRepository studentRepository;

    @RequestMapping("test1")
    public List<Object[]> test1() {
        return studentRepository.findSugangCount();
    }

    @RequestMapping("test2")
    public List<Student> test2() {
        return studentRepository.findByLectureTite("자료구조");
    }

    @RequestMapping("test3")
    public List<Student> test3() {
        return studentRepository.findByProfessorIdOrProfessorName(0, "이몽룡");
    }

    @RequestMapping("test4")
    public List<Student> test4() {
        return studentRepository.findByProfessorNameOrProfessorId(null, 13);
    }

}
```

4) SugangController.java

src/main/java/net/skhu/controller/SugangController.java

```java
package net.skhu.controller;

import java.util.List;
import jakarta.transaction.Transactional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import net.skhu.entity.Sugang;
import net.skhu.repository.SugangRepository;

@RestController
@RequestMapping("sugang")
public class SugangController {

    @Autowired SugangRepository sugangRepository;

    @Transactional
    @RequestMapping("test1")
    public List<Sugang> test1() {
        sugangRepository.udateGradeByLectureId(3, "C0", "D+");
        return sugangRepository.findByLectureIdOrderByGradeDesc(3);

    }

    @Transactional
    @RequestMapping("test2")
    public List<Sugang> test2() {
        sugangRepository.udateGradeByLectureId(3, "D+", "C0");
        return sugangRepository.findByLectureIdOrderByGradeDesc(3);
    }

}
```

@Transactional
@Modifying 어노테이션이 붙은 메소드를 호출하는 코드를 포함하는 메소드에
@Transactional 어노테이션을 반드시 붙여야 한다.
이 어노테이션에 대한 설명은 transaction 강의노트에 있다.

5) 실행

아래 URL 들을 차례로 실행해 보자.

```
http://localhost:8088/student/test1
http://localhost:8088/student/test2
http://localhost:8088/student/test3
http://localhost:8088/student/test4

http://localhost:8088/sugang/test1
http://localhost:8088/sugang/test2
```

6) StudentRepository.java 수정

src/main/java/net/skhu/repository/StudentRepository.java

```
1  package net.skhu.repository;
2
3  import java.util.List;
4  import org.springframework.data.jpa.repository.JpaRepository;
5  import org.springframework.data.jpa.repository.Query;
6  import net.skhu.entity.Student;
7
8  public interface StudentRepository extends JpaRepository<Student, Integer>  {
9
10          @Query("SELECT  s.studentNo,  s.name,  SIZE(s.sugangs)  FROM  Student  s  ORDER  BY
11 SIZE(s.sugangs) DESC")
12     List<Object[]> findSugangCount();
13
14     @Query("SELECT s FROM Student s JOIN s.sugangs g WHERE g.lecture.title = ?1")
15     List<Student> findByLectureTite(String title);
16
17     @Query("""
18          SELECT s
19          FROM Student s
20          WHERE EXISTS (SELECT g.lecture
21                        FROM s.sugangs g
22                            WHERE g.lecture.professor.id = ?1 OR g.lecture.professor.name = ?2)
23 """)
24     List<Student> findByProfessorIdOrProfessorName(int id, String name);
25
26      List<Student> findBySugangsLectureProfessorIdOrSugangsLectureProfessorName(int id, String
27 name);
28
29     @Query("""
30          SELECT s
31          FROM Student s
32          WHERE EXISTS (SELECT g.lecture
33                        FROM s.sugangs g
34                         WHERE g.lecture.professor.id = :id OR g.lecture.professor.name = :name)
   """)
     List<Student> findByProfessorNameOrProfessorId(String name, int id);

   }
```

(줄23)의 findByProfessorIdOrProfessorName 메소드를,
JPA query creation 기능을 활용하여, 다음과 같이 구현할 수도 있다.

```
List<Student>  findBySugangsLectureProfessorIdOrSugangsLectureProfessorName(int  id,  String
name);
```

Student 엔터티에는 sugagns 속성이 있고,
Sugang 엔터티에는 lecture 속성이 있고,
Lecture 엔터티에는 professor 속서이 있고,
Professor 엔터티에는 id, name 속성이 있다.

8.부록 - 고급 SQL

김진우 학생이 수강하는 강좌를 전부 다 듣는 학생 목록

#1

```
use student2;

SELECT s.id studentId, s.name studentName, e.id lectureId, e.title
    FROM student s
      JOIN sugang g ON g.studentId = s.id
      JOIN lecture e ON g.lectureId = e.id
```

```
1 •  use student2;
2
3    SELECT s.id studentId, s.name studentName, e.id lectureId, e.title
4        FROM student s
5          JOIN sugang g ON g.studentId = s.id
6          JOIN lecture e ON g.lectureId = e.id
7
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

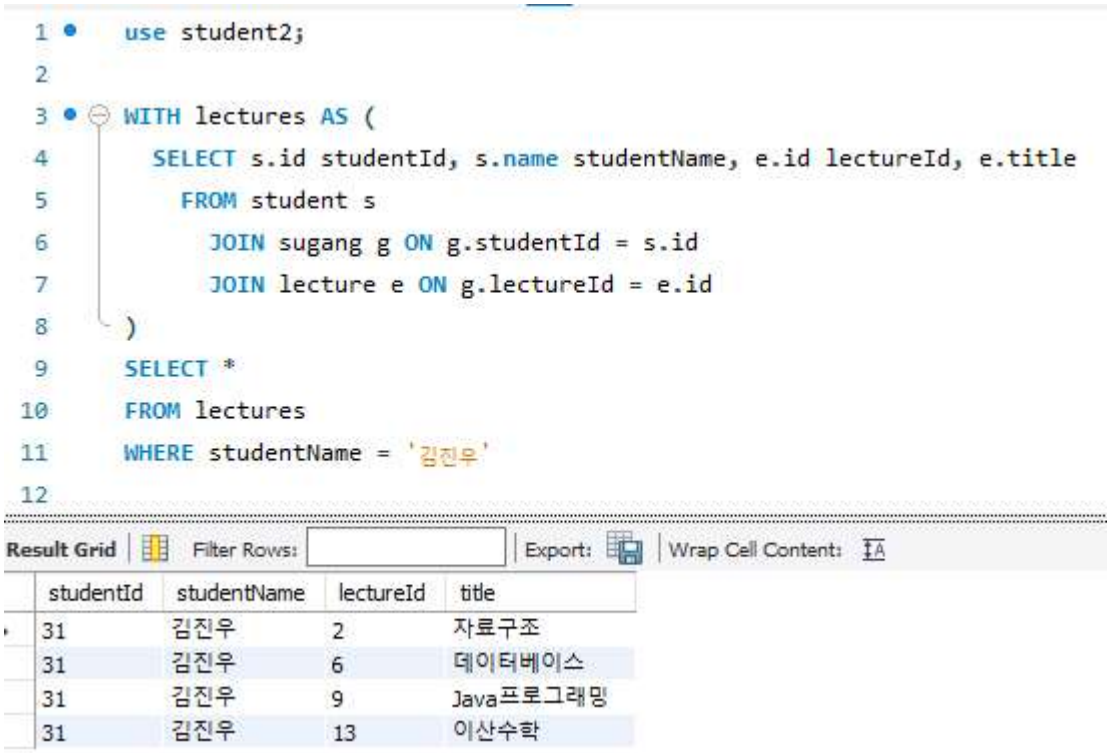
studentId	studentName	lectureId	title
1	최하은	1	Java프로그래밍
2	김진영	1	Java프로그래밍
3	나철진	1	Java프로그래밍
4	이익수	1	Java프로그래밍
5	이제문	1	Java프로그래밍
6	김영우	1	Java프로그래밍
7	주한요	1	Java프로그래밍
8	김숙홍	1	Java프로그래밍
9	홍영수	1	Java프로그래밍
10	박원종	1	Java프로그래밍
11	변준호	1	Java프로그래밍
12	고희정	1	Java프로그래밍
13	신철대	1	Java프로그래밍
14	서윤정	1	Java프로그래밍
15	오화순	1	Java프로그래밍
16	박재인	1	Java프로그래밍
17	최현복	1	Java프로그래밍
18	임봉영	1	Java프로그래밍
19	장영만	1	Java프로그래밍
20	박주용	1	Java프로그래밍
21	황하지	1	Java프로그래밍
22	유환치	1	Java프로그래밍
23	성희지	1	Java프로그래밍
24	박철용	1	Java프로그래밍
25	이주연	1	Java프로그래밍
26	김철지	1	Java프로그래밍
27	이화상	1	Java프로그래밍
28	백영규	1	Java프로그래밍
29	김화인	1	Java프로그래밍
30	김만기	1	Java프로그래밍
30	김만기	2	자료구조
31	김진우	2	자료구조
32	박남영	2	자료구조
33	이식동	2	자료구조
34	박하정	2	자료구조
35	박룡성	2	자료구조
36	곽구인	2	자료구조
37	김균진	2	자료구조

#2

```
use student2;

WITH lectures AS (
  SELECT s.id studentId, s.name studentName, e.id lectureId, e.title
    FROM student s
      JOIN sugang g ON g.studentId = s.id
      JOIN lecture e ON g.lectureId = e.id
)
SELECT *
FROM lectures
WHERE studentName = '김진우'
```

김진우 학생이 수강하는 강좌 목록



```
1  use student2;
2
3  WITH lectures AS (
4    SELECT s.id studentId, s.name studentName, e.id lectureId, e.title
5      FROM student s
6        JOIN sugang g ON g.studentId = s.id
7        JOIN lecture e ON g.lectureId = e.id
8  )
9  SELECT *
10 FROM lectures
11 WHERE studentName = '김진우'
12
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

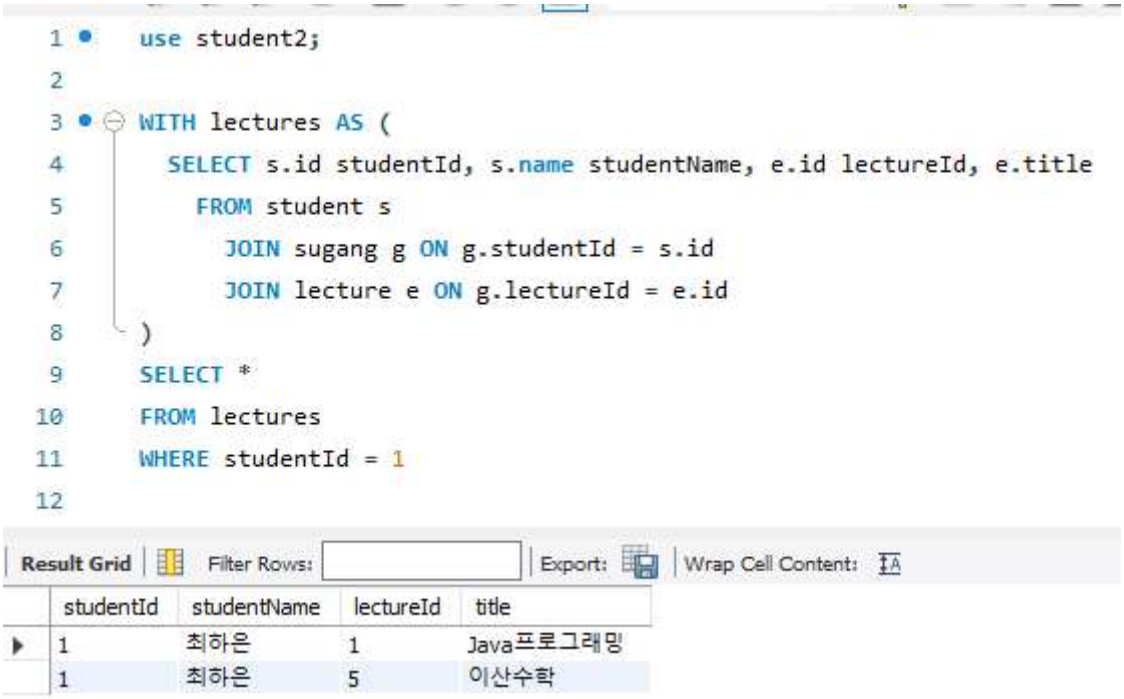
studentId	studentName	lectureId	title
31	김진우	2	자료구조
31	김진우	6	데이터베이스
31	김진우	9	Java프로그래밍
31	김진우	13	이산수학

#3

```
use student2;

WITH lectures AS (
  SELECT s.id studentId, s.name studentName, e.id lectureId, e.title
    FROM student s
      JOIN sugang g ON g.studentId = s.id
      JOIN lecture e ON g.lectureId = e.id
)
SELECT *
FROM lectures
WHERE studentId = 1
```

id = 1 인 학생이 수강하는 강좌 목록



```
use student2;

WITH lectures AS (
  SELECT s.id studentId, s.name studentName, e.id lectureId, e.title
    FROM student s
      JOIN sugang g ON g.studentId = s.id
      JOIN lecture e ON g.lectureId = e.id
)
SELECT *
FROM lectures
WHERE studentId = 1
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

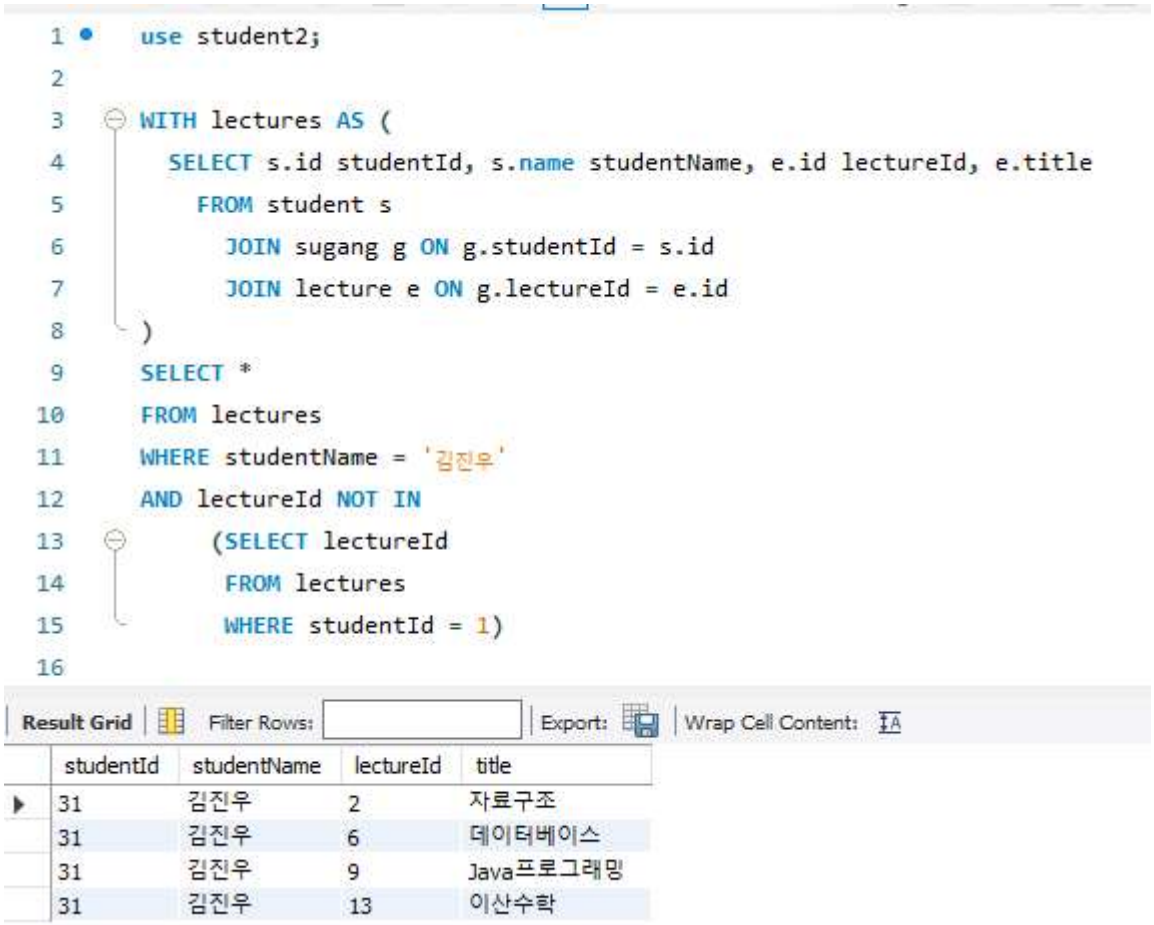
studentId	studentName	lectureId	title
1	최하은	1	Java프로그래밍
1	최하은	5	이산수학

#4

```
use student2;

WITH lectures AS (
  SELECT s.id studentId, s.name studentName, e.id lectureId, e.title
    FROM student s
      JOIN sugang g ON g.studentId = s.id
      JOIN lecture e ON g.lectureId = e.id
)
SELECT *
FROM lectures
WHERE studentName = '김진우'
AND lectureId NOT IN
     (SELECT lectureId
      FROM lectures
      WHERE studentId = 1)
```

김진우 학생이 수강하는 강좌 중에서,
id = 1 인 학생이 수강하지 않는 강좌들의 목록



```
1  use student2;
2
3  WITH lectures AS (
4    SELECT s.id studentId, s.name studentName, e.id lectureId, e.title
5      FROM student s
6        JOIN sugang g ON g.studentId = s.id
7        JOIN lecture e ON g.lectureId = e.id
8  )
9  SELECT *
10 FROM lectures
11 WHERE studentName = '김진우'
12 AND lectureId NOT IN
13      (SELECT lectureId
14       FROM lectures
15       WHERE studentId = 1)
16
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

studentId	studentName	lectureId	title
31	김진우	2	자료구조
31	김진우	6	데이터베이스
31	김진우	9	Java프로그래밍
31	김진우	13	이산수학

#5 - 완성

```
use student2;

WITH lectures AS (
  SELECT s.id studentId, s.name studentName, e.id lectureId, e.title
    FROM student s
      JOIN sugang g ON g.studentId = s.id
      JOIN lecture e ON g.lectureId = e.id
)
SELECT *
FROM Student s
WHERE
NOT EXISTS
  (SELECT *
   FROM lectures
   WHERE studentName = '김진우'
   AND lectureId NOT IN
     (SELECT lectureId
      FROM lectures
      WHERE studentId = s.id))
```

각각의 학생 s 에 대해서
김진우 학생이 수강하는 강좌 중에서,
s 학생이 수강하지 않는 강좌가
없으면 그 학생이 조회 결과에 포함된다.